

# Assignment 2: Art Gallery Triangulation with voids

Bothamolla Charan(24B0966)  
Chunchikala SreeCharan(24B0914)

April 3, 2026

## Contents

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                             | <b>2</b> |
| 1.1      | The Guard Bound for Polygons with Holes . . . . .               | 2        |
| <b>2</b> | <b>Topological Model: Multiply Connected Polygons</b>           | <b>2</b> |
| 2.1      | Differentiating Solid vs. Void (Boundary Orientation) . . . . . | 2        |
| 2.2      | Enforcement of Topological Invariants . . . . .                 | 2        |
| 2.3      | Solid-vs-Void Membership Rule . . . . .                         | 3        |
| <b>3</b> | <b>Detailed Data Structures and Memory Layout</b>               | <b>3</b> |
| 3.1      | Polygon Node (Single Ring) . . . . .                            | 3        |
| 3.2      | Floor-Plan Node (Complete Domain) . . . . .                     | 3        |
| <b>4</b> | <b>Helper Geometric Functions</b>                               | <b>3</b> |
| <b>5</b> | <b>Algorithm Implementation Pipeline</b>                        | <b>4</b> |
| <b>6</b> | <b>Main Algorithms and Pseudocode</b>                           | <b>5</b> |
| 6.1      | Hole Merging ( <code>merge_holes</code> ) . . . . .             | 5        |
| 6.2      | Ear Clipping Triangulation ( <code>ear_clip</code> ) . . . . .  | 6        |
| 6.3      | Graph 3-Colouring ( <code>three_colour</code> ) . . . . .       | 7        |
| <b>7</b> | <b>Correctness and Robustness Proofs</b>                        | <b>7</b> |
| <b>8</b> | <b>Final Total Time Complexity Analysis</b>                     | <b>8</b> |
| <b>9</b> | <b>Conclusion</b>                                               | <b>8</b> |

# 1 Introduction

This report details the design, implementation, and analysis of a robust computational geometry algorithm to solve the Art Gallery Problem for a complex floor plan . The gallery is modeled as a multiply connected planar polygon  $P$  in  $\mathbb{R}^2$  containing an outer boundary and  $h$  interior holes . The objective is to partition this space into simple triangles and calculate the strict mathematical upper bound of "point guards" required to achieve total visibility .

## 1.1 The Guard Bound for Polygons with Holes

The classical Art Gallery Theorem states that  $\lfloor n/3 \rfloor$  guards are sufficient for a simple polygon . However, the presence of holes complicates visibility . For a polygon with  $n$  total vertices and  $h$  holes, the strict theoretical upper bound of guards is  $\lfloor (n + 2h)/3 \rfloor$  .

**Theorem 1** (Bjorling-Sachs and Souvaine, 1995). *For any polygon  $P$  with  $n$  vertices and  $h$  holes,  $\lfloor (n + 2h)/3 \rfloor$  point guards are always sufficient and sometimes necessary to guard  $P$  .*

*Proof.* By creating bridge edges (cuts) to connect each hole to the outer boundary, we convert  $P$  into a degenerate simple polygon . Each bridge adds exactly two vertices to the boundary sequence . Bridging  $h$  holes adds  $2h$  vertices, resulting in a total count of  $n' = n + 2h$  . Applying Chvátal's bound to the simply-connected result gives  $\lfloor (n + 2h)/3 \rfloor$  .  $\square$

## 2 Topological Model: Multiply Connected Polygons

The art-gallery domain  $P$  is modeled as a planar region with one solid outer boundary and zero or more interior voids (holes):

$$P = \Omega \setminus (H_1 \cup H_2 \cup \dots \cup H_k)$$

where  $\Omega$  is the closed solid exterior ring and each  $H_i$  is an open void region.

### 2.1 Differentiating Solid vs. Void (Boundary Orientation)

We differentiate solid interiors from void spaces via **Winding Order**. The signed area of a ring encodes its topological role:

| Ring Type          | Winding | Signed Area | Meaning           |
|--------------------|---------|-------------|-------------------|
| Outer ( $\Omega$ ) | CCW (+) | $> 0$       | <b>SOLID</b> Area |
| Hole ( $H_i$ )     | CW (-)  | $< 0$       | <b>VOID</b> Space |

A CCW-oriented boundary has its enclosed solid material to its **LEFT**. A CW-oriented boundary has its void space to its **RIGHT**. This follows the non-zero winding-number fill rule: CCW rings ADD solid area, while CW rings SUBTRACT void area.

### 2.2 Enforcement of Topological Invariants

The program enforces four critical invariants to ensure geometric validity:

- **(T1) Orientation:** Guaranteed by `ensure_ccw()` for outer rings and `ensure_cw()` for holes.
- **(T2) Containment:** Every hole  $H_i$  must lie strictly inside  $\Omega$ . Verified implicitly during `merge_holes()`.
- **(T3) Non-Overlap:** Holes do not intersect each other or the outer boundary.
- **(T4) No Shared Edges:** No edge is shared between rings, as zero-width solid regions make triangulation undefined.

### 2.3 Solid-vs-Void Membership Rule

A point  $q$  belongs to the **SOLID INTERIOR** of  $P$  if and only if:

$$\text{wind}(q, \Omega) = +1 \quad \text{AND} \quad \text{wind}(q, H_i) = 0 \quad \forall i$$

## 3 Detailed Data Structures and Memory Layout

The program utilizes two primary node structures to maintain the topological separation between solid and void regions.

### 3.1 Polygon Node (Single Ring)

Represents either the outer boundary or a specific hole.

- **vertices**: A list of  $(x, y)$  coordinate tuples. The list is implicitly closed.
- **winding**: An integer field caching the topological state:
  - **+1**: CCW (Outer solid boundary).
  - **-1**: CW (Hole / void boundary).
  - **0**: Degenerate (Zero area).

### 3.2 Floor-Plan Node (Complete Domain)

The high-level structure representing the entire gallery.

- **outer**: Exactly **one** CCW Polygon node (Solid).
- **holes**: A list of zero or more CW Polygon nodes (Void).

This structural separation ensures that downstream functions never have to guess the topological role of a ring; the "outer" key is always the additive solid, and the "holes" list contains the subtractive voids.

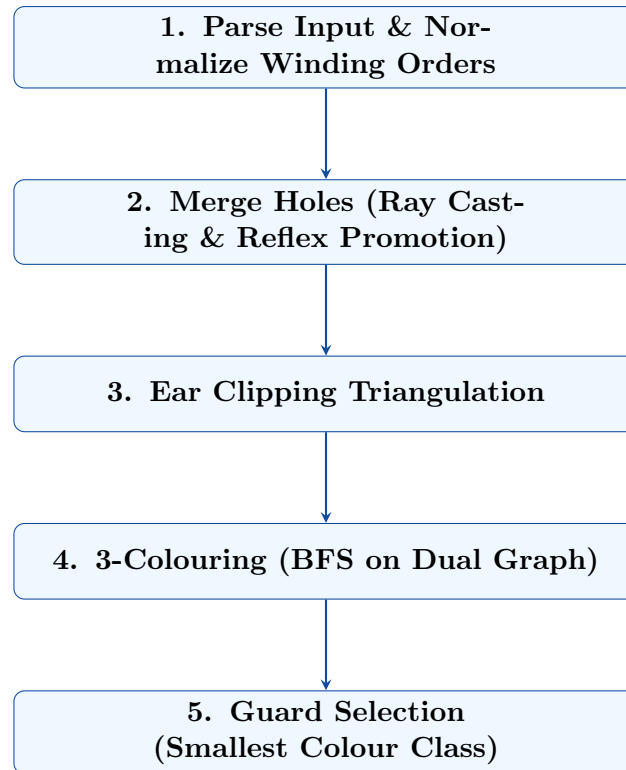
## 4 Helper Geometric Functions

Before main execution, low-level geometric primitives compute spatial relationships :

1. **polygon\_area(ring)**: Calculates signed area using the Shoelace Formula. **Complexity:  $O(V)$**  where  $V$  is the number of vertices .
2. **ensure\_ccw/cw(ring)**: Enforces orientation by reversing the array if the area sign does not match the target topology. **Complexity:  $O(V)$**  .
3. **cross(o, a, b)**: 2D cross product of vectors OA and OB. Used to determine if a turn is left, right, or collinear. **Complexity:  $O(1)$**  .
4. **point\_in\_triangle(p, a, b, c)**: Determines if point  $p$  lies inside  $\triangle abc$  using cross products. **Complexity:  $O(1)$**  .
5. **intersect\_ray\_segment**: Casts a horizontal ray using a half-open interval  $[y_{min}, y_{max})$  to handle exact collinear vertex strikes safely. **Complexity:  $O(1)$**  .
6. **is\_ear(prev, curr, nxt, ring)**: Determines if a vertex is a valid ear (convex and no other reflex vertices lie inside the triangle). **Complexity:  $O(n)$**  .

## 5 Algorithm Implementation Pipeline

The algorithm follows a strict sequence to transform complex input into a guard layout:



## 6 Main Algorithms and Pseudocode

### 6.1 Hole Merging (*merge\_holes*)

---

**Algorithm 1** Merge Holes into Outer Boundary

---

```
1: function MERGEHOLES(outer_verts, holes_verts)
2:   merged  $\leftarrow$  outer_verts
3:   Sort holes_verts by rightmost x-coordinate descending.
4:   for each hole in sorted_holes do
5:     M  $\leftarrow$  rightmost vertex of hole
6:     Cast horizontal ray rightward from M to find closest intersection I on edge  $p_0 - p_1$ .
7:     Vcand  $\leftarrow$  endpoint of  $p_0 - p_1$  with larger x-coordinate.
8:     Reflex Promotion: Check all vertices  $q \in$  merged inside  $\triangle MIV_{cand}$ .
9:     if q is a reflex vertex then
10:       Vcand  $\leftarrow$  q (choosing the one minimizing the angle with the ray).
11:     end if
12:     Splice merged at Vcand, insert  $M \rightarrow hole \rightarrow M \rightarrow V_{cand}$ .
13:   end for
14:   return merged
15: end function
```

---

**Detailed Explanation:** This algorithm connects interior holes to the outer boundary to form a single continuous ring. By sorting holes by their rightmost x-coordinate, we ensure that new bridges do not intersect previously merged holes. The **Reflex Promotion** step is critical: if any reflex vertex of the outer boundary obstructs the visibility between the hole and the target edge, we "promote" that reflex vertex to be the bridge endpoint, ensuring the bridge never crosses the boundary. **Complexity:**  $O(h \cdot n)$  where  $h$  is the number of holes and  $n$  is total vertices. Each hole requires ray casting ( $O(n)$ ) and reflex checks ( $O(n)$ )

## 6.2 Ear Clipping Triangulation (ear\_clip)

---

**Algorithm 2** Optimized Ear Clipping

---

```
1: function EARCLIP(ring)
2:   active  $\leftarrow$  doubly-linked list of vertex indices
3:   ear_status  $\leftarrow$  boolean array of size n
4:   for i = 0 to n - 1 do
5:     ear_status[i]  $\leftarrow$  IsEar(i, active)
6:   end for
7:   while |active| > 3 do
8:     Find vertex c  $\in$  active where ear_status[c] == True.
9:     Output triangle (prev[c], c, next[c]).
10:    Remove c from active.
11:    Re-evaluate IsEar for prev[c] and next[c] in O(n).
12:   end while
13:   Output final 3 vertices as the last triangle.
14: end function
```

---

**Detailed Explanation:** Ear clipping iteratively removes a convex vertex (an "ear") that contains no other reflex vertices in its triangle. By using a doubly-linked circular list, we maintain the neighborhood of each vertex after a clip in  $O(1)$  time. Crucially, when an ear is clipped, only the status of its two immediate neighbors can change. This limits the  $O(n)$  IsEar check to just two vertices per iteration. **Complexity:**  $O(n^2)$ . Initializing ear status takes  $O(n^2)$ . The clip loop runs  $n$  times, with each iteration performing  $O(n)$  re-evaluations.

### 6.3 Graph 3-Colouring (`three_colour`)

---

**Algorithm 3** 3-Colouring via Triangle BFS

---

```
1: function THREECOLOUR(triangles, coords)
2:   colour_map  $\leftarrow$  Hash map of index  $\rightarrow$  colour
3:   adj  $\leftarrow$  Map of shared edges to adjacent triangles
4:   queue  $\leftarrow$  Queue containing first triangle
5:   Colour vertices of first triangle with colours  $\{0, 1, 2\}$ .
6:   while queue is not empty do
7:     curr_tri  $\leftarrow$  queue.dequeue()
8:     for each next_tri sharing an edge with curr_tri do
9:       if next_tri is uncoloured then
10:        Let (u, v) be the shared edge (already coloured).
11:        Let w be the uncoloured vertex in next_tri.
12:        colour_map[w]  $\leftarrow$   $3 - (\text{colour\_map}[u] + \text{colour\_map}[v])$ 
13:        Enqueue next_tri.
14:       end if
15:     end for
16:   end while
17:   return colour_map
18: end function
```

---

**Detailed Explanation:** The dual graph of a triangulated simple polygon is a tree. We use BFS to traverse this tree. For any new triangle in the BFS, two vertices on its shared edge are already coloured. Since the sum of colours  $\{0, 1, 2\}$  is 3, the third vertex is uniquely assigned the remaining colour. This ensures no triangle contains duplicate colours. **Complexity:**  $O(n)$ . Building the adjacency map takes  $O(n)$ , and BFS visits each triangle exactly once.

## 7 Correctness and Robustness Proofs

Our implementation guarantees valid geometric execution through several logical safeguards:

1. **Topological Validity:** By defining bridge edges using ray-casting, we connect holes to the outer boundary. The bridge edges are traversed twice (inwards and outwards), meaning their net contribution to the topological area is zero. This creates a single continuous boundary containing  $n + 2h$  vertices that is geometrically valid for triangulation.
2. **Numerical Robustness:** The program uses a half-open interval  $[y_{min}, y_{max})$  combined with an epsilon ( $\epsilon = 1e - 9$ ) tolerance for ray-segment intersection. This ensures that if a ray strikes a vertex exactly, it is only counted for one edge, preventing double-counting or missed intersections.
3. **Reflex-Vertex Promotion Guarantee:** Standard bridging might create a cut that intersects the outer boundary. Our algorithm checks if any reflex vertex of the outer boundary lies within the triangle formed by the hole vertex, the intersection point, and the target vertex. Promoting the obstructing reflex vertex to be the bridge target guarantees a bridge edge that never intersects the boundary.

4. **3-Colouring Sufficiency:** Because the dual graph of a triangulated polygon is a tree, it is always 3-colourable . By the Pigeonhole Principle, at least one of the three colour sets must contain  $\leq \lfloor (n + 2h)/3 \rfloor$  vertices . Placing guards at these vertices ensures every triangle is watched by at least one guard .

## 8 Final Total Time Complexity Analysis

The final total time complexity is the sum of the complexities of each component in the pipeline:

| Pipeline Stage              | Complexity     | Primary Operation                               |
|-----------------------------|----------------|-------------------------------------------------|
| Input Parsing & Orientation | $O(n)$         | Shoelace formula and array reversals            |
| Hole Merging                | $O(h \cdot n)$ | Ray casting and reflex promotion                |
| Ear Clipping Triangulation  | $O(n^2)$       | Initial ear precomputation and neighbor updates |
| Graph 3-Colouring           | $O(n)$         | Dual graph BFS traversal                        |
| Guard Class Selection       | $O(n)$         | Pigeonhole calculation on vertex classes        |

**Total Complexity:**

$$T(n, h) = O(n) + O(h \cdot n) + O(n^2) + O(n) = \mathbf{O(n^2 + hn)}$$

Given that  $h < n$  (as holes are formed by vertices within the total count), the complexity is effectively dominated by the Ear Clipping phase, resulting in a robust  $\mathbf{O(n^2)}$  algorithm . This meets the efficiency requirements for complex polygons with hundreds of vertices .

## 9 Conclusion

The security architecture for the art gallery successfully utilizes a multiply connected polygon model to represent solid boundaries and internal courtyards . By implementing a topological merging strategy, we transformed a complex structure with holes into a simply connected domain with  $n + 2h$  vertices . The optimized  $O(n^2)$  ear clipping triangulation combined with linear-time 3-colouring provides a computationally feasible and mathematically rigorous method for determining guard placement . This approach guarantees that  $\lfloor (n + 2h)/3 \rfloor$  guards are sufficient to achieve 360-degree visibility over the entire walkable space, satisfying the strict requirements of the Art Gallery Problem .